

O problema central é compreender e representar, de forma visual e analítica, como os desenvolvedores interagem dentro de um projeto colaborativo no GitHub. A proposta é criar uma ferramenta capaz de transformar essas interações, como comentários, merges e fechamentos de issues, em um grafo que mostre as conexões entre os participantes. A partir disso, busca-se analisar como a colaboração acontece, quem são os membros mais ativos ou influentes e quais grupos se formam naturalmente dentro do projeto. Em outras palavras, o desafio é usar a teoria dos grafos para traduzir a dinâmica humana de trabalho em equipe em uma estrutura que possa ser estudada e entendida de forma técnica e visual.

Escolhemos o repositório awesome-javascript por se tratar de uma das listas mais completas e conhecidas da comunidade JavaScript, reunindo ferramentas, frameworks e bibliotecas amplamente utilizadas no desenvolvimento moderno. Essa curadoria facilita a pesquisa e o aprendizado sobre boas práticas, padrões de código e tecnologias consolidadas, o que contribui diretamente para o aprimoramento técnico e para a construção de projetos mais bem estruturados. Além disso, a organização e a atualização constante do repositório o tornam uma excelente referência para estudos e desenvolvimento acadêmico. Assim se tornando uma boa opção para realizarmos a modelagem do trabalho prático.

A coleta de dados foi realizada por meio da API pública do GitHub, utilizando uma classe chamada GithubClient, desenvolvida para buscar e organizar informações do repositório awesome-javascript. Essa abordagem permitiu reunir dados de forma automatizada e precisa, garantindo que todas as interações fossem coletadas diretamente da fonte.

Foram extraídos dados de issues, pull requests e comentários, incluindo quem criou, comentou, revisou, fechou ou fez o merge dessas interações. Cada uma dessas ações foi interpretada como uma forma de colaboração entre os usuários.

Essas informações foram então transformadas em arestas dentro de um grafo direcionado, onde cada usuário é representado como um nó. Por exemplo, um comentário gera uma conexão entre o autor do comentário e o criador da issue ou PR. Essa estrutura permite visualizar e analisar como os colaboradores se conectam, destacando os mais participativos e as relações de cooperação dentro do projeto.

A modelagem do grafo foi pensada para representar as interações entre os colaboradores do repositório. Cada usuário é um nó, e cada interação é uma aresta direcionada entre dois nós. Foram criados três grafos parciais, um para comentários, outro para fechamentos e outro para merges, que depois foram integrados em um grafo único com pesos que indicam a intensidade da colaboração. Os pesos seguem a importância de cada ação: comentário (2), abertura de issue comentada (3), revisão/aprovação de PR (4) e merge (5). Assim, o grafo final mostra de forma clara quem mais colabora e como essas conexões acontecem dentro do projeto.

O desenvolvimento da solução será iniciado com a coleta e organização dos dados. Nessa fase, serão extraídas informações sobre issues, pull requests e comentários, que depois serão tratadas para representar os usuários e suas interações. Em seguida, será implementada a estrutura de grafos, composta pelas classes AbstractGraph, AdjacencyMatrixGraph e AdjacencyListGraph, responsáveis por manipular as conexões entre os colaboradores.

Na segunda etapa, as interações coletadas serão integradas ao grafo de forma ponderada, seguindo os pesos definidos conforme a relevância de cada tipo de ação (comentários, revisões, fechamentos e merges). Serão aplicadas métricas de análise, como centralidade e densidade, para identificar os principais colaboradores e padrões de

cooperação dentro do projeto. Por fim, o grafo será exportado, possibilitando a visualização e interpretação dos resultados, enquanto todo o desenvolvimento será versionado no GitHub para garantir organização e controle do progresso.